GCC129 · SISTEMAS DISTRIBUÍDOS · 2026/2

# CampoConecta

Parte 2 — Arquitetura · projeto documentado, contratos e decisões

- Conecta a **agricultura familiar** à merenda escolar
- Quatro serviços, dois clientes, dois BFFs, um banco por serviço
- Documentação verificável — **sem sistema em execução** nesta etapa

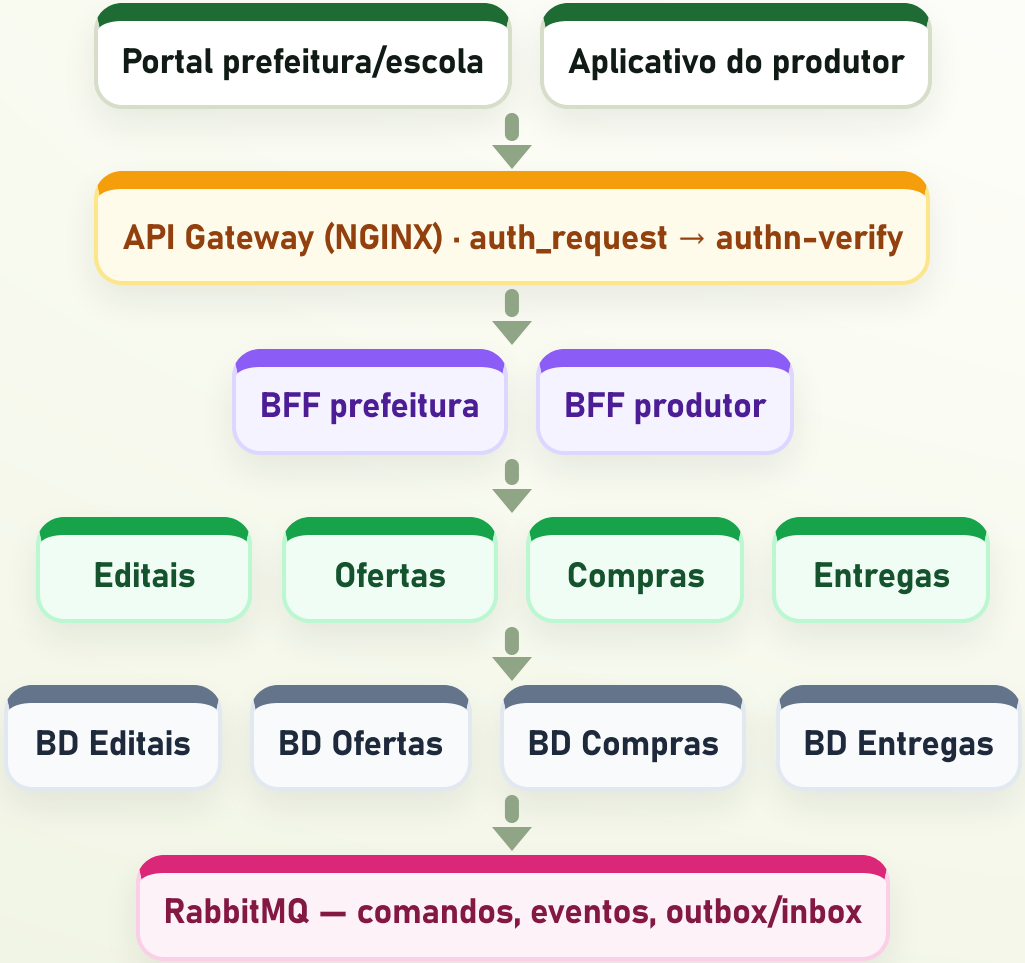
**Nesta etapa entregamos o projeto: serviços, contratos, dados e os fluxos distribuídos.**

# O problema, em uma frase

- A Lei 11.947/2009 exige **30% do FNDE** em produtos da agricultura familiar
- Prefeituras não conseguem operar a cota: gestor despreparado + produtor desorganizado
- O CampoConecta digitaliza o caminho: **edital** → **proposta** → **alocação** → **entrega** → **pagamento**

**Existe uma obrigação legal e uma lacuna operacional: é essa lacuna que o sistema resolve.**

# Arquitetura em uma imagem




**A única porta de entrada é o gateway; cada serviço só escreve no seu banco.**

# Quatro serviços e por que esse corte

- **Editais** — demanda: prefeitura, escola, edital, saldo do item
- **Ofertas** — produção: produtor, oferta, reserva
- **Compras** — compromisso: proposta, plano, compra, SAGA, pagamento
- **Entregas** — logística: capacidade, agendamento, recebimento

**Cortamos por ciclo de vida e regra própria, não por tabela — por isso não existe um "serviço de cadastros".**

# Contratos REST

- OpenAPI 3.1: **4 serviços + 2 BFFs**, todos sobre `/api/v1`
- Status com significado: **201** criado, **202** assíncrono, **409** conflito, **503** dependência fora
- Versionamento: evolução compatível mantém v1; quebra exige nova versão
- **HATEOAS** no recurso Compra: `_links` variam por estado e papel

**O contrato é executável: validamos os seis arquivos com Redocly e um script de cobertura.**

# Gateway e dois BFFs

## Gateway (NGINX)

Entrada única, TLS, `auth_request` → JWT validado no `authn-verify`, rate limit, CORS, correlation-id.
**Não** agrega e **não** aplica regra de negócio.

## Dois BFFs, dois clientes

**Prefeitura:** tela ampla, página de 50, objetos aninhados, dashboard.
**Produtor:** mobile, cursor de 10, payload achatado, ETag.

**Mesmo recurso, respostas diferentes: o portal vê contexto; o aplicativo vê só o essencial.**

# Um banco por serviço — e o outbox

- Quatro instâncias **PostgreSQL**, usuário próprio, **sem FK entre bancos**
- Dado de outro serviço vira **snapshot** com `snapshotAt`, nunca fonte de verdade
- Reservas por `UPDATE` condicional: `saldo >= quantidade`
- **Dual write eliminado**: entidade e linha do outbox na mesma transação
- Consumidor grava `messageId` na inbox → entrega repetida, efeito único

**Não usamos transação distribuída: outbox resolve o dual write e a inbox garante idempotência.**

# SAGA: confirmar a compra

- **Orquestrada** em Compras — estado em `saga_compra` e `saga_operacao`
- Homologação cria N compras → **uma SAGA por compra**
- Passos: **AlocarDemanda** (Editais) → **ReservarOferta** (Ofertas) → **AgendarEntrega** (Entregas) → concluir
- Falha no agendamento: **LiberarOferta** + **LiberarDemanda** → compra CANCELADA

**Quatro serviços, quatro passos e três compensações: a compra só cancela depois de desfazer tudo.**

# CQRS em Compras

### Sem projeção

Acompanhar uma compra exigiria **4 chamadas** — Editais, Ofertas, Compras e Entregas — e remontar a SAGA no cliente.

### Com a projeção

Uma consulta em `acompanhamento_compra`, atualizada por evento, com `versao` para não regredir.

- Defasagem aceitável: **até 5 s** (meta declarada, ainda não medida)
- Reconstrução a partir das tabelas de escrita — **sem replay do broker**

**A projeção é só leitura: saldo, autorização e pagamento continuam decididos no serviço dono.**

# Onde verificar

- README: tabela de **cobertura da rubrica** — critério, peso e documento
- `docs/parte2/01..06`: serviços, dados/outbox, CQRS, gateway/BFFs, SAGA e mensagens
- `docs/adr/`: sete decisões com alternativas e consequências
- Validação: `redocly lint` nos seis contratos + `verificar-contratos.cjs`

**Tudo é rastreável a partir do README: o critério, o documento e a evidência.**